

* Dependency Injection using Annotation Based Configuration →

- i) Constructor Injection.
- ii) Setter Injection.
- iii) Field Injection.

① Address.java (common dependency class)

```
package com.example;
```

```
import org.springframework.stereotype.Component;
```

@Component

```
public class Address {
```

```
    public String getAddress() {
```

```
        return "Pune, Maharashtra";
```

```
    }
```

Date: _____

□ M □ T □ W □ T □ F □ S □ S

② Constructor Injection (Student.java) ^{CS}

package com.example;



Auto import for
@Component
@Autowired
in IDEA

@Component ^{CS}
public class Student {

private Address address;

@Autowired ^{CS}
public Student(Address address) {

 this.address = address;
}

public void show() {

 System.out.println("Constructor Injection" +
 address.getAddress());
}

}

③ Setter Injection (StudentSI.java)

package com.example

@Component

public class StudentSI {

private Address address;

@Autowired

public void setAddress(Address address) {

this.address = address;

public void show() {

System.out.println("setter Injection" +
address.getAddress());

}

}

Date: _____

□ M □ T □ W □ T □ F □ S □ S

④ Field Injection (StudentFI.java)

```
package com.example;
```

```
@Component
```

```
public class StudentFI {
```

```
@Autowired
```

```
private Address address
```

```
public void show() {
```

```
    System.out.println("Field Injection " +  
        address.getAddress());
```

```
}
```

```
}
```

⑤ Configuration (AppConfig.java)

```
package com.example;
```

```
@Configuration
```

```
@ComponentScan(basePackages = "com.example")
```

```
public class AppConfig {
```

```
}
```

⑥ Main class (MainApp.java)

```
package com.example;
```

```
public class MainApp {
```

```
    public static void main(String[] args) {
```

```
        ApplicationContext context = new  
            AnnotationConfigApplicationContext  
            (AppConfig.class);
```

```
        StudentCI studentCI = context.getBean  
            (StudentCI.class);
```

```
        or (StudentCI) context.getBean  
            ("studentCI"); (in this we  
            need to  
            cast)
```

```
        studentCI.show();
```

```
        StudentSI studentSI = context.getBean  
            (StudentSI.class);
```

```
        studentSI.show();
```

```
        StudentFI studentFI = context.getBean  
            (StudentFI.class);
```

```
        studentFI.show();
```

```
}  
}
```

@Qualifier Annotation →

The @Qualifier annotation in Spring is used to resolve dependency injection conflicts when multiple beans of the same type are present in the Spring container.

Why is @Qualifier needed?

When there are multiple beans of the same type, Spring doesn't know which one to inject, and you'll get error like:

NonUniqueBeanDefinitionException:

To solve this, we use @Qualifier to tell Spring which specific bean you want to inject.

Example without Qualifier (Error Case) →

@Component

```
public class Honda implements Car {
```

```
    public void drive() {
```

```
        System.out.println("Driving Honda...");
```

```
    }
```

@Component

```
public class Toyota implements Car {  
    public void drive() {  
        System.out.println("Driving Toyota");  
    }  
}
```

@Component

```
public class Traveller {
```

@Autowired

```
private Car car; // X Error: Multiple beans  
of type Car.
```

(Now let's understand how can we resolve bean conflict using @Qualifier Annotation)

Syntax →

@Autowired

```
@Qualifier("beanName")  
private ClassType field;
```

Example fixing problem using @Qualifier

@Component

public class Traveller {

@Autowired

@Qualifier("honda")

private Car car;

public void startJourney() {

car.drive();

}

}

{ name of bean
to inject
Here it is class
name but first
letter is small

{ (If we name bean customly using
@Component("myHonda"))

then we need to write

@Qualifier("myHonda") while Autowiring)

}

What is application.properties →

In Spring the application.properties file is used to externalize configuration - like database URLs, port numbers, or custom values - so that we didn't have to hard-code them in Java code.

Build in Properties (write in application.properties file)

server.port = 8081

spring.database.url=jdbc:mysql://.....

spring.application.name = Demo App.

etc.

Custom Properties (application.properties file)

app.title = Meeting Scheduler

app.version = 1.0.0

Access Custom Properties in code

@Component

```
public class AppInfo {
```

```
    @value("${app.title}")
```

```
    private String title;
```

```
    @value("${app.version}")
```

```
    private String version;
```

```
    public void propertiesInfo() {
```

```
        System.out.println("Title: " + title);
```

```
        System.out.println("Version: " + version);
```

```
    }
```

```
}
```

* @Bean Annotation

The @Bean Annotation is used to manually define a Bean in Java-based Configuration class. which is annotated with @Configuration

If we create bean manually then no need to write @Component on class.

It is alternative to @Component, used when you want more control over how the object is created.

Example →

@Configuration

public class AppConfig {

@Bean

public Address address() {

return new Address("Pune", "MH");
}

@Bean

public Student student() {

return new Student(address());
// Injecting address bean manually.
}

Explanation →

i) address() & student() methods define & return beans.

ii) These beans are managed by Spring just like @Component beans.

* What is Spring MVC :->

- A Spring MVC is sub framework of Spring framework which is used to build a web application.
- It is build on the top of Servlet API.
- It follows the Model-View-Controller design pattern.
- It implements all the basic features of a core Spring framework like Inversion of Control, Dependency Injection

* Why Spring MVC :->

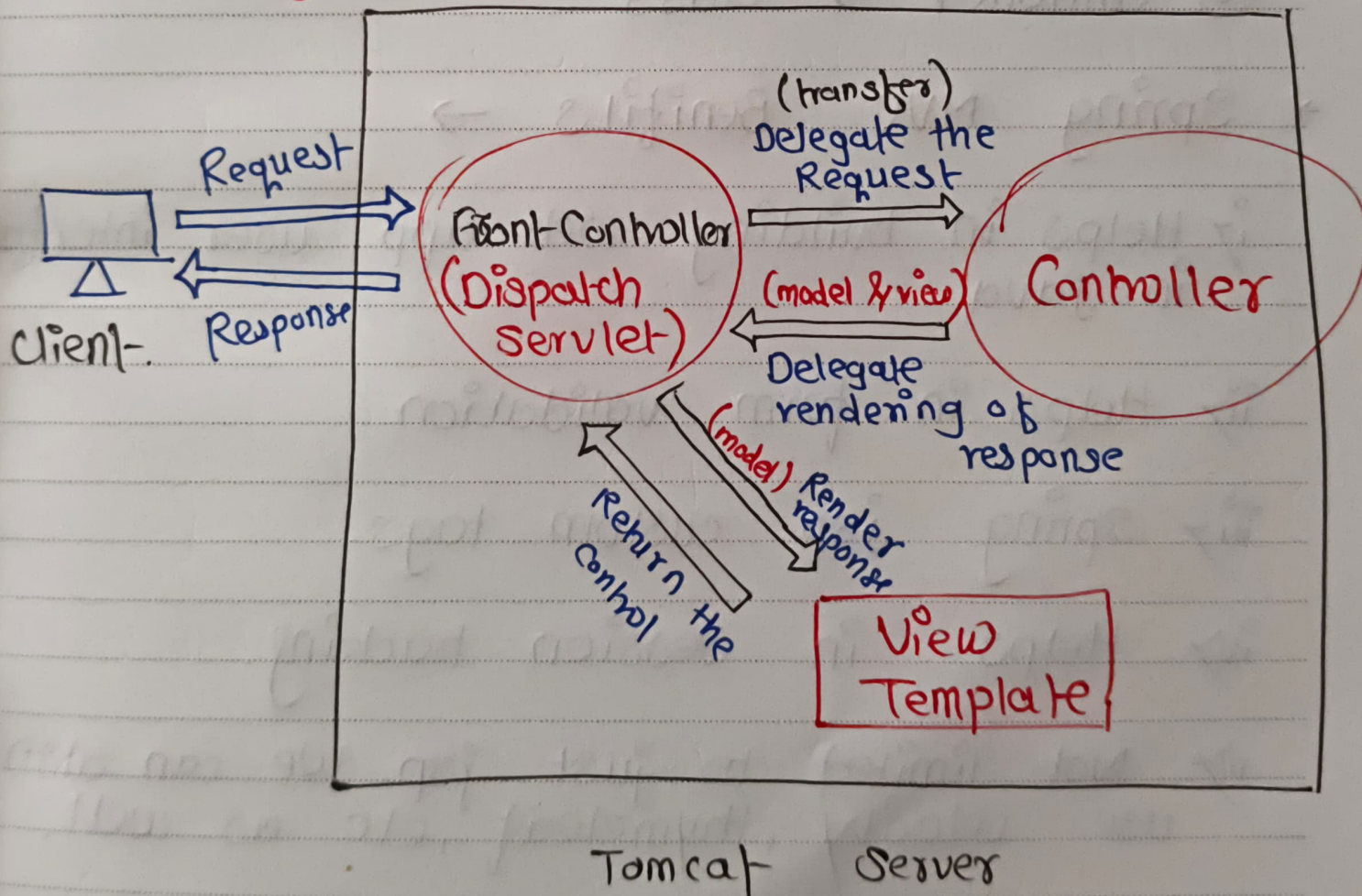
- Separate each role i.e Model, view Controller.
- Powerful Configuration.
- It is sub framework of Spring framework so that we can use its features.
- Spring MVC is flexible, easy to test and more features.

* MVC Design Pattern →

Model → Data
 View → Presents data to user
 Controller → Interface between model and view

- Way to Organize the code in our Application

* * * Working of Spring MVC :->



Note →

i) Front Controller will be DispatcherServlet given us by Spring.

ii) We only need to create MVC

① Model → Data (Collection of Data), POJO

② View → HTML, CSS, JSP

③ Controller → class

* Spring MVC Benefits →

i) Helps in building web app user interface in Java

ii) Helps in form validation

iii) Spring JSP custom tags.

iv) Helps in session tracking.

v) Not limited to just JSP. We can also use Velocity, Thymeleaf, etc as well.